

Abstract

This document specifies the experimental protocol for the **gen.2** study of neural network-based track extrapolation in the LHCb experiment. The study trains multi-layer perceptrons (MLPs), physics-informed neural networks (PINNs), and Runge-Kutta-structured PINNs (RK_PINNs) to approximate the fourth-order Runge-Kutta (RK4) numerical extrapolator that propagates charged-particle states through the LHCb dipole magnetic field. Nine model configurations are trained on 50e6 RK4-generated tracks spanning $\Delta z \in [25, 10000]$ mm and $p \in [1, 200]$ GeV/ c . The protocol addresses two research questions: (RQ1) whether GPU-deployable models (≤ 64 kB) achieve accuracy competitive with RK4 for integration into Allen HLT1, and (RQ2) whether physics-informed training improves accuracy-per-parameter relative to purely data-driven training. All data generation parameters, model architectures, loss functions, training hyperparameters, evaluation metrics, known confounds, and reproducibility provisions are specified in full.

Experimental Protocol: Second-Generation Neural Track Extrapolation for the LHCb Allen HLT1 Trigger

G. Scriven

Maastricht University, UHasselt, Nikhef

April 23, 2026

1 Introduction

The LHCb detector [1] reconstructs the trajectories of charged particles produced in proton–proton collisions at the Large Hadron Collider. Track reconstruction requires propagating particle states through the inhomogeneous LHCb dipole magnetic field from one detector element to another—a procedure called *track extrapolation*. The production extrapolator is a fourth-order Runge-Kutta (RK4) integrator [2] that evaluates the Lorentz force equation at each integration step, with the magnetic field obtained by trilinear interpolation of a measured three-dimensional field map [3].

In the upgraded LHCb detector operating at Run 3 and beyond, the first-level software trigger (HLT1) runs entirely on GPUs [4]. The Allen framework executes real-time pattern recognition at the full 30 MHz bunch-crossing rate. Track extrapolation—called $\mathcal{O}(10^6)$ times per event—is a significant contributor to the per-event processing time. Replacing the RK4 integrator with a neural network that can be evaluated efficiently in GPU registers or constant memory offers a potential throughput improvement, provided the accuracy degradation is acceptable for the downstream Kalman filter [5].

This document defines the protocol for **gen_2**, the second generation of this investigation. The first generation (**gen_1**) established feasibility: a two-layer MLP with architecture [64, 64] achieved a mean position error of 0.49 mm and was deployable within the 64 kB CUDA `__constant__` memory limit. However, **gen_1** had two shortcomings: (i) training data used $\Delta z_{\min} = 100$ mm, which does not cover the 25–30 mm spacing between VELO silicon pixel sensors, and (ii) the larger models that achieved better accuracy exceeded the 64 kB deployment constraint.

gen_2 addresses both issues by extending the Δz range down to 25 mm and constraining all model architectures to ≤ 64 kB. It further introduces physics-informed training as a mechanism to improve accuracy without increasing model size.

1.1 Research Questions

RQ1 — Engineering viability. Are GPU-deployable models (≤ 64 kB) accurate enough for integration into the Allen HLT1 workflow? Specifically, does the MLP extrapolator achieve position and slope accuracy competitive with the RK4 ground truth, particularly at the short Δz values (25–100 mm) relevant to VELO track fitting?

RQ2 — Physics-informed training. For a given architecture size, does embedding the Lorentz force ordinary differential equation (ODE) as a soft constraint during training (PINN, RK_PINN) yield a model that generalises better—measured by accuracy-per-parameter—compared to purely data-driven (MLP) training?

2 Data Generation Protocol

2.1 Ground-Truth Integrator

Training data are generated by a fixed-step-size RK4 integrator (`experiments/gen.2/utis/rk4_propagator.py`) that propagates the five-dimensional state vector $\mathbf{y} = (x, y, t_x, t_y, q/p)$ through the LHCb magnetic field from an initial longitudinal position z_0 to $z_f = z_0 + \Delta z$. The integration step size is $h = 5 \text{ mm}$, chosen to be well within the stability region of RK4 for the field gradients encountered in the LHCb tracking volume [2]. Momentum (q/p) is conserved along the trajectory (no material interactions).

2.2 Magnetic Field Map

The magnetic field is evaluated by trilinear interpolation of the measured field map `twodip.rtf`, which tabulates B_x, B_y, B_z on an $81 \times 81 \times 146$ Cartesian grid:

- $x, y \in [-4000, 4000] \text{ mm}$ at 100 mm spacing,
- $z \in [-500, 14000] \text{ mm}$ with variable spacing,
- peak $|B_y| \approx 1.03 \text{ T}$ at $z \approx 5007 \text{ mm}$.

The same interpolation module (`experiments/gen.2/utis/magnetic_field.py`) is used for data generation (NumPy backend) and for physics-informed training (PyTorch backend with `torch.nn.functional.grid_sample`), ensuring consistency between the ground truth and the PDE residual computation.

2.3 Sampling Strategy

Each training sample consists of a randomly drawn initial state and step size. The parameter ranges and sampling distributions are:

$$z_0 \sim \mathcal{U}(0, 14000 - \Delta z) \text{ mm}, \quad (1)$$

$$\Delta z \sim \mathcal{U}(25, 10000) \text{ mm}, \quad (2)$$

$$p \sim \text{LogU}(1, 200) \text{ GeV}/c, \quad (3)$$

$$x_0 \sim \mathcal{U}(-300, 300) \text{ mm}, \quad (4)$$

$$y_0 \sim \mathcal{U}(-250, 250) \text{ mm}, \quad (5)$$

$$t_{x,0} \sim \mathcal{U}(-0.3, 0.3), \quad (6)$$

$$t_{y,0} \sim \mathcal{U}(-0.25, 0.25), \quad (7)$$

where $\mathcal{U}$ denotes uniform sampling and LogU denotes log-uniform (uniform in $\log p$). The charge sign is fixed to $q = -1$ (MagDown polarity). Tracks that diverge beyond $|x|$ or $|y| > 5000 \text{ mm}$, or whose slopes exceed $|t_x|$ or $|t_y| > 2.0$, are rejected as unphysical.

The extension of $\Delta z_{\min}$ from 100 mm (`gen.1`) to 25 mm is motivated by the VELO sensor pitch: the Kalman filter propagates states between consecutive VELO planes separated by 25–30 mm, and the neural extrapolator must handle these short steps.

2.4 Input and Output Representation

The neural network input is a six-dimensional vector:

$$\mathbf{x} = (x_0, y_0, t_{x,0}, t_{y,0}, q/p, \Delta z), \quad (8)$$

with units (mm, mm, –, –, MeV^{-1} , mm). The output is the four-dimensional extrapolated state:

$$\mathbf{y} = (x_f, y_f, t_{x,f}, t_{y,f}), \quad (9)$$

omitting q/p (conserved) and z_f (deterministic: $z_f = z_0 + \Delta z$).

2.5 Dataset Construction

A total of 50e6 tracks are generated in 2000 parallel HTCondor batches of 25000 tracks each (`condor/submit_datagen.sub`). Each batch uses a deterministic seed:

$$s_i = 42 + i \times 7919, \quad i = 0, 1, \dots, 1999, \quad (10)$$

where 7919 is a large prime chosen to space seeds far apart in the pseudorandom number generator’s period. After all batches complete, the individual `.npz` files are merged (`data/merge_batches.py`) into a single dataset `train_50M_dz25.npz` containing arrays:

- $\mathbf{X} \in \mathbb{R}^{N \times 6}$: input states,
- $\mathbf{Y} \in \mathbb{R}^{N \times 4}$: output states,
- $\mathbf{P} \in \mathbb{R}^N$: momenta (GeV/ c),
- $\mathbf{Z} \in \mathbb{R}^N$: initial z_0 positions.

The dataset is split by random permutation (seed 42) into:

Split	Fraction	Samples
Train	80%	40e6
Validation	10%	5e6
Test	10%	5e6

3 Physics: Equations of Motion

A charged particle traversing the LHCb magnetic field $\mathbf{B} = (B_x, B_y, B_z)$ follows the Lorentz force law. In the LHCb z -parameterisation [2], where z (the beam axis) serves as the independent variable, the equations of motion are:

$$\frac{dx}{dz} = t_x, \quad (11)$$

$$\frac{dy}{dz} = t_y, \quad (12)$$

$$\frac{dt_x}{dz} = \kappa \mathcal{N} [t_x t_y B_x - (1 + t_x^2) B_y + t_y B_z], \quad (13)$$

$$\frac{dt_y}{dz} = \kappa \mathcal{N} [(1 + t_y^2) B_x - t_x t_y B_y - t_x B_z], \quad (14)$$

where

$$\kappa = \frac{q}{p} \times c_{\text{light}}, \quad \mathcal{N} = \sqrt{1 + t_x^2 + t_y^2}, \quad (15)$$

and $c_{\text{light}} = 2.99792458 \times 10^{-4}$ mm/(MeV · T) is the conversion factor from SI to LHCb internal units. The dominant field component is B_y (vertical), which causes horizontal (x) bending. The slopes $t_x = dx/dz$ and $t_y = dy/dz$ are dimensionless direction tangents.

Equations (11)–(14) define the ODE system $d\mathbf{y}/dz = \mathbf{F}(\mathbf{y}, z; \mathbf{B})$ that underpins both the RK4 data generation and the physics-informed loss terms.

4 Model Specifications

All architectures inherit from a common base class that provides z -score input normalisation $\tilde{\mathbf{x}} = (\mathbf{x} - \boldsymbol{\mu}_x)/\boldsymbol{\sigma}_x$ and output denormalisation $\hat{\mathbf{y}} = \boldsymbol{\sigma}_y \tilde{\mathbf{y}} + \boldsymbol{\mu}_y$, computed from the training set. All trainable parameters use Xavier uniform initialisation [6] unless otherwise stated.

4.1 MLP (Multi-Layer Perceptron)

The MLP is a standard feedforward network:

$$\hat{\mathbf{y}} = W_{L+1} \sigma(W_L \cdots \sigma(W_1 \tilde{\mathbf{x}} + b_1) \cdots + b_L) + b_{L+1}, \quad (16)$$

where σ is the SiLU (Swish) activation [7] $\sigma(x) = x \cdot \text{sigmoid}(x)$, and output denormalisation is applied after the final linear layer. Dropout is set to zero for all `gen_2` configurations.

The training loss is purely data-driven:

$$\mathcal{L}_{\text{MLP}} = \frac{1}{4N} \sum_{i=1}^N \sum_{k=1}^4 \left(\frac{\hat{y}_{ik} - y_{ik}}{\sigma_k} \right)^2, \quad (17)$$

where σ_k is the standard deviation of output component k in the training set ($k \in \{x, y, t_x, t_y\}$).

Component-weighted MSE. The network output is denormalised before loss computation. Because the physical magnitudes of the four output components span many orders of magnitude— $|\Delta x| \sim \mathcal{O}(1) \text{ mm}$, $|\Delta t_x| \sim \mathcal{O}(10^{-3})$ —a plain MSE in physical space assigns the position components $\sim 10^6$ times more gradient weight than the slopes. Dividing each component error by its training-set standard deviation σ_k is equivalent to computing MSE in normalised (z-score) space, which restores gradient balance across all four components. This is consistent with the non-dimensionalisation approach recommended for multi-scale PINN losses [12, 13].

4.2 PINN (Physics-Informed Neural Network)

The PINN uses a residual formulation that guarantees satisfaction of the initial condition (IC) by construction. The architecture decomposes the prediction into a physics-based baseline (straight-line extrapolation) plus a learned correction modulated by the fractional propagation distance $\zeta = z/\Delta z \in [0, 1]$:

$$\hat{\mathbf{y}}(\zeta) = \mathbf{y}_{\text{base}}(\zeta) + \zeta \cdot \boldsymbol{\delta}_{\boldsymbol{\theta}}(\mathbf{x}_0), \quad (18)$$

where

$$\mathbf{y}_{\text{base}}(\zeta) = \begin{pmatrix} x_0 + t_{x,0} \zeta \Delta z \\ y_0 + t_{y,0} \zeta \Delta z \\ t_{x,0} \\ t_{y,0} \end{pmatrix}, \quad (19)$$

and $\boldsymbol{\delta}_{\boldsymbol{\theta}}$ is the output of a correction network. At $\zeta = 0$, $\hat{\mathbf{y}} = \mathbf{y}_{\text{base}}(0) = (x_0, y_0, t_{x,0}, t_{y,0})$, satisfying the IC identically. The correction $\boldsymbol{\delta}_{\boldsymbol{\theta}}$ is predicted by an encoder that takes only the five-dimensional initial state $\mathbf{x}_0 = (x_0, y_0, t_{x,0}, t_{y,0}, q/p)$ as input (excluding Δz), followed by a linear correction head:

$$\boldsymbol{\delta}_{\boldsymbol{\theta}} = W_{\text{head}} \phi_L \circ \cdots \circ \phi_1(\tilde{\mathbf{x}}_0) + b_{\text{head}}, \quad (20)$$

where each $\phi_l(\mathbf{h}) = \tanh(W_l \mathbf{h} + b_l)$. The $\tanh$ activation is used instead of SiLU to ensure smooth derivatives for automatic differentiation of the PDE residual via `torch.autograd.grad` with `create_graph=True`.

The PINN training loss is:

$$\mathcal{L}_{\text{PINN}} = \mathcal{L}_{\text{data}} + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}} + \lambda_{\text{pde}} \mathcal{L}_{\text{pde}}, \quad (21)$$

where each term is non-dimensionalised by the output standard deviations σ_k from the training set, following the gradient-balancing prescription of Wang et al. [12]:

- $\mathcal{L}_{\text{data}}$ is the component-weighted endpoint MSE (Eq. 17), identical across all three model types;

- $\mathcal{L}_{\text{ic}}$ is the initial-condition loss, computed in the same normalised space:

$$\mathcal{L}_{\text{ic}} = \frac{1}{4N} \sum_{i=1}^N \sum_{k=1}^4 \left(\frac{\hat{y}_{ik}(\zeta=0) - y_{0,ik}}{\sigma_k} \right)^2. \quad (22)$$

Because of the residual formulation (Eq. 18), $\hat{\mathbf{y}}(\zeta=0) \equiv \mathbf{y}_0$ exactly, so $\mathcal{L}_{\text{ic}} = 0$ by construction and serves as a sanity check.

- $\mathcal{L}_{\text{pde}}$ is the PDE residual loss. Define the raw residual vector at collocation point k and sample i :

$$\mathbf{r}_{ik} = \left. \frac{\partial \hat{\mathbf{y}}}{\partial z} \right|_{\zeta_k} - \mathbf{F}(\hat{\mathbf{y}}(\zeta_k), z_0 + \zeta_k \Delta z; \mathbf{B}), \quad (23)$$

where $\mathbf{F}$ is the right-hand side of the ODE system (Eqs. 11–14). The raw residuals for each component have different physical dimensions and magnitudes: $r^{(x)} \sim \mathcal{O}(t_x) \sim 0.1$ (dimensionless), while $r^{(t_x)} \sim \mathcal{O}(\kappa B) \sim 10^{-5} \text{ mm}^{-1}$. To bring all four components to $\mathcal{O}(1)$, we non-dimensionalise by the characteristic derivative scale $\sigma_k/\Delta z_i$:

$$\bar{r}_{ik,j} = \frac{r_{ik,j}}{\sigma_j/\Delta z_i}, \quad j \in \{x, y, t_x, t_y\}, \quad (24)$$

which follows from introducing $\bar{s}_j \equiv s_j/\sigma_j$, $\bar{z} \equiv z/\Delta z$, so that $d\bar{s}_j/d\bar{z} = (\Delta z/\sigma_j)(ds_j/dz)$ is $\mathcal{O}(1)$ when the trajectory is well-described by the training distribution. The PDE loss is then:

$$\mathcal{L}_{\text{pde}} = \frac{1}{4n_c N} \sum_{k=1}^{n_c} \sum_{i=1}^N \sum_{j=1}^4 \bar{r}_{ik,j}^2, \quad (25)$$

evaluated at $n_c = 8$ collocation points uniformly spaced in $\zeta \in [0.1, 1.0]$. Because Δz_i varies per sample, the non-dimensionalisation automatically applies looser absolute tolerances for long propagation paths while maintaining the same relative tolerance.

Note on PDE residual composition. All four ODE equations are enforced: two kinematic ($dx/dz = t_x$, $dy/dz = t_y$) and two Lorentz (Eqs. 13–14). Although the kinematic residuals provide redundant gradient signal on the slopes (which are already trained by $\mathcal{L}_{\text{data}}$), they enforce internal consistency of the trajectory—i.e. that the network’s position and slope outputs are mutually consistent along the full path, not just at the endpoints. This is standard practice in the PINN literature [11].

Note on clamping. No output clamping is applied before computing the PDE residual. Earlier code used `torch.clamp` to restrict predictions to a physical range, but this breaks the `autograd` computation graph (zero gradient outside the clamp bounds), silently zeroing the PDE loss for out-of-range predictions.

At each collocation point, the PDE residual requires: one forward pass to obtain $\hat{\mathbf{y}}(\zeta_k)$, four calls to `torch.autograd.grad` to obtain $\partial \hat{y}_j/\partial \zeta$ for $j = 1, \dots, 4$, a conversion $\partial \hat{\mathbf{y}}/\partial z = (\partial \hat{\mathbf{y}}/\partial \zeta)/\Delta z$, a magnetic field evaluation at $(x_k, y_k, z_0 + \zeta_k \Delta z)$, and the Lorentz residual from Eqs. (11)–(14).

The correction head weights are initialised uniformly in $[-0.01, 0.01]$ with zero bias, so that the network begins at the straight-line baseline. A learnable scalar `field_scale` is included but initialised to 1.

4.3 RK_PINN (Runge-Kutta Physics-Informed Neural Network)

The RK_PINN is a multi-stage architecture inspired by the RK4 numerical integrator [9, 10]. A shared backbone extracts features from the normalised six-dimensional input; four stage-specific heads then predict the state at intermediate positions $\zeta_s \in \{0.25, 0.5, 0.75, 1.0\}$:

$$\mathbf{f} = \text{Backbone}(\tilde{\mathbf{x}}), \quad (26)$$

$$\hat{\mathbf{y}}_s = \text{Head}_s([\mathbf{f}; \zeta_s]), \quad s = 1, \dots, 4, \quad (27)$$

$$\hat{\mathbf{y}} = \sum_{s=1}^4 w_s \hat{\mathbf{y}}_s, \quad w_s = \frac{e^{\alpha_s}}{\sum_{s'} e^{\alpha_{s'}}}, \quad (28)$$

where $[\cdot; \cdot]$ denotes concatenation, α_s are learnable logits initialised to $\log(c_s)$ with c_s from the RK4 Butcher tableau $(c_1, c_2, c_3, c_4) = (1, 2, 2, 1)/6$, and the softmax ensures $\sum w_s = 1$.

The backbone consists of the first $L - 1$ layers of the specified architecture; each stage head consists of a hidden layer of size d_L (the last element of the `hidden_dims` list) followed by a linear output layer to four dimensions. For `hidden_dims` = [96, 96], the backbone has one layer (Linear(6, 96) + tanh) and each head has Linear(97, 96) + tanh + Linear(96, 4).

The training loss mirrors the PINN:

$$\mathcal{L}_{\text{RK_PINN}} = \mathcal{L}_{\text{data}} + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}} + \lambda_{\text{pde}} \mathcal{L}_{\text{pde}}. \quad (29)$$

All three terms use the *same non-dimensionalisation* as the PINN (Eqs. 17, 22, 25), ensuring that λ_{pde} has a consistent physical meaning across model types. The IC loss is computed as the normalised MSE between the initial state and the average of all four stage-head predictions evaluated at $\zeta = 0$. The PDE residual is computed at each of the four stage positions using the non-dimensional scaling $\sigma_k/\Delta z_i$ (Eq. 24). Critically, the backbone features are *detached* before being passed to the stage heads for the PDE gradient computation—gradients propagate only through the stage heads, preventing interference between the data loss (which trains the full network) and the PDE loss (which trains only the heads).

4.4 Parameter Counts and Memory Footprint

Table 1 reports the trainable parameter counts computed directly from the architecture code. All parameters are stored in single-precision (float32, 4 B each), and the memory footprint is $4 \times N_{\text{params}}$ bytes.

Table 1: Trainable parameter counts and memory footprint for each model configuration, computed from the architecture source code. The CUDA `--constant--` memory limit is 64 kB (16384 float32 parameters).

Config	Type	Dims	Params	KB
<code>mlp/tiny</code>	MLP	[64, 64]	4868	19.0
<code>mlp/small</code>	MLP	[96, 96]	10372	40.5
<code>mlp/medium</code>	MLP	[112, 112]	13892	54.3
<code>pin*/</code>	PINN	[96, 96]	10277 ^a	40.1
<code>rk_pinn/*</code>	RK_PINN	[96, 96]	39860 ^b	155.7

^a Encoder input is 5-dimensional (no Δz); includes one learnable `field_scale` parameter.

^b Four separate stage heads each contain Linear(97, 96) + Linear(96, 4); total is $672 + 4 \times 9796 + 4 = 39860$. This *exceeds* the 64 kB `--constant--` memory limit and is therefore relevant only to RQ2 (scientific interest), not RQ1 (engineering deployment).

Remark. The PINN encoder takes a 5-dimensional input (initial state without Δz), yielding 96 fewer first-layer weights than the MLP; it also includes one additional scalar parameter (`field_scale`). The RK_PINN, by design, replicates the output head four times, resulting in approximately four times the parameter count of a single-head model with the same hidden dimensions. This means the “same architecture size” comparison between MLP, PINN, and RK_PINN at `hidden_dims` = [96, 96] is *not* a controlled comparison of parameter count. Section 9 discusses this confound in detail.

5 Experimental Design

5.1 Configuration Matrix

Table 2 specifies all nine training runs. Each configuration is defined by a JSON file in `experiments/gen_2/configs/`.

Table 2: Complete configuration matrix for the `gen_2` experiment. All runs use seed 42, weight decay 10^{-4} , maximum 300 epochs, gradient clipping at 1.0, and cosine annealing with 5-epoch linear warmup. Physics loss weights λ_{pde} and λ_{ic} apply only to PINN and RK_PINN models.

Config	Type	Hidden	Act.	Batch	lr	Pat.	λ_{pde}	λ_{ic}	n_c	Field
<code>mlp/tiny</code>	MLP	[64, 64]	SiLU	4096	10^{-3}	30	—	—	—	—
<code>mlp/small</code>	MLP	[96, 96]	SiLU	4096	10^{-3}	30	—	—	—	—
<code>mlp/medium</code>	MLP	[112, 112]	SiLU	4096	10^{-3}	30	—	—	—	—
<code>pinn/lam0.1</code>	PINN	[96, 96]	tanh	1024	5×10^{-4}	40	0.1	0.1	8	interp.
<code>pinn/lam1.0</code>	PINN	[96, 96]	tanh	1024	5×10^{-4}	40	1.0	0.1	8	interp.
<code>pinn/lam10.0</code>	PINN	[96, 96]	tanh	1024	5×10^{-4}	40	10.0	0.1	8	interp.
<code>rk_pinn/lam0.1</code>	RK_PINN	[96, 96]	tanh	1024	5×10^{-4}	40	0.1	1.0	4 [†]	interp.
<code>rk_pinn/lam1.0</code>	RK_PINN	[96, 96]	tanh	1024	5×10^{-4}	40	1.0	1.0	4 [†]	interp.
<code>rk_pinn/lam10.0</code>	RK_PINN	[96, 96]	tanh	1024	5×10^{-4}	40	10.0	1.0	4 [†]	interp.

[†] RK_PINN evaluates the PDE residual at the four fixed stage positions $\zeta \in \{0.25, 0.5, 0.75, 1.0\}$ rather than at n_c uniformly spaced collocation points.

5.2 Comparisons and Controls

The nine configurations enable the following structured comparisons:

MLP size scaling (RQ1). `mlp/tiny` $\rightarrow$ `mlp/small` $\rightarrow$ `mlp/medium` tests how accuracy improves with model capacity within the 64 kB budget. The controlled variables are activation (SiLU), batch size (4096), and learning rate (10^{-3}).

MLP vs. PINN at matched architecture (RQ2). `mlp/small` [96, 96] vs. `pinn/lam*` [96, 96] compares data-driven and physics-informed training. The hidden dimensions match, but several variables are *not* controlled (see Sec. 9).

MLP vs. RK_PINN at matched hidden dimensions (RQ2). `mlp/small` [96, 96] vs. `rk_pinn/lam*` [96, 96]. The hidden dimensions match but the RK_PINN has $\approx 4\times$ more parameters due to four stage heads (Table 1).

PINN λ_{pde} sweep (RQ2). $\lambda_{\text{pde}} \in \{0.1, 1.0, 10.0\}$ at fixed $\lambda_{\text{ic}} = 0.1$ tests the sensitivity of the PINN to the physics loss weight.

RK_PINN λ_{pde} sweep (RQ2). $\lambda_{\text{pde}} \in \{0.1, 1.0, 10.0\}$ at fixed $\lambda_{\text{ic}} = 1.0$ for the RK_PINN architecture.

PINN vs. RK_PINN (RQ2). At each λ_{pde} value, the PINN and RK_PINN can be compared to assess whether the multi-stage RK structure provides additional benefit. This comparison is confounded by the parameter count difference.

6 Training Protocol

6.1 Optimiser and Schedule

All models are trained with AdamW [8] ($\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay = 10^{-4}). The learning rate follows a cosine annealing schedule with a 5-epoch linear warmup:

$$\text{lr}(t) = \begin{cases} \text{lr}_0 \cdot t/5, & t \leq 5, \\ \frac{\text{lr}_0}{2} \left(1 + \cos\left(\pi \cdot \frac{t-5}{T-5}\right) \right), & t > 5, \end{cases} \quad (30)$$

where $T = 300$ is the maximum number of epochs. Gradient norms are clipped to 1.0.

6.2 Early Stopping

Training terminates early if the validation loss does not improve by more than $\delta_{\min} = 10^{-7}$ for a number of consecutive epochs specified by the patience parameter: 30 for MLPs, 40 for PINN and RK_PINN. The extended patience for physics-informed models accounts for the slower convergence typically observed when balancing multiple loss terms.

6.3 Checkpointing

Model weights are saved every 25 epochs and at the best validation loss. The best-epoch checkpoint is used for all evaluation.

6.4 Infrastructure

Training is executed on the Nikhef HTCondor cluster (`taai-007.nikhef.nl`) via the submission file `condor/submit.sub`, which dispatches one GPU job per configuration. GPU nodes provide either NVIDIA Tesla V100-PCIE-32GB or NVIDIA L40S accelerators. The `+JobCategory = "long"` policy allows extended wall-clock time. The software environment is:

- Conda environment `TE` (Python 3.10),
- PyTorch 2.9.1 with CUDA 12.8,
- NumPy 2.2.6, MLflow 3.10.1.

Full dependency pinning is recorded in `environment.yml` and `requirements.txt`.

6.5 Experiment Tracking

All runs are logged to MLflow under the experiment name `gen_2_track_extrapolation`. Logged artefacts include: training and validation loss per epoch (total and per-component for PINN/RK_PINN), learning rate schedule, model configuration, best-epoch checkpoint, and test-set evaluation metrics. TensorBoard summaries are written in parallel.

7 Evaluation Protocol

All metrics are computed on the held-out test set (5e6 samples, disjoint from training and validation data).

7.1 Primary Metrics

Position error.

$$\|\Delta \mathbf{r}\| = \sqrt{(\hat{x}_f - x_f)^2 + (\hat{y}_f - y_f)^2}, \quad (31)$$

reported as mean, standard deviation, 95th percentile, and 99th percentile.

Slope error.

$$\|\Delta \mathbf{s}\| = \sqrt{(\hat{t}_{x,f} - t_{x,f})^2 + (\hat{t}_{y,f} - t_{y,f})^2}, \quad (32)$$

reported as mean and 95th percentile.

Per-component errors. $|\Delta x|$, $|\Delta y|$, $|\Delta t_x|$, $|\Delta t_y|$ reported as means.

7.2 Secondary Metrics

Jacobian (transport matrix). The 4×5 Jacobian

$$J_{ij} = \frac{\partial \hat{y}_i}{\partial x_{0,j}}, \quad i \in \{x, y, t_x, t_y\}, \quad j \in \{x, y, t_x, t_y, q/p\}, \quad (33)$$

is computed via `torch.autograd.grad` on 1000 randomly selected test samples. The Frobenius norm $\|J\|_F$ is reported as mean and standard deviation.

Remark on Jacobian discriminative power. In `gen_1`, all models produced $\|J\|_F \approx 5650 \pm 4200$ with minimal inter-model variation. This suggests the metric is dominated by the large- Δz regime where the Jacobian magnitude is inherently large. A Δz -binned analysis (Sec. 8) may be more informative.

7.3 Success Criteria

RQ1. A model is considered viable for Allen HLT1 deployment if:

1. it fits within 64 kB (16384 float32 parameters),
2. its mean position error $\langle \|\Delta \mathbf{r}\| \rangle < 0.5 \text{ mm}$ on the full test set,
3. its 95th-percentile position error $< 2 \text{ mm}$,
4. its mean slope error $\langle \|\Delta \mathbf{s}\| \rangle < 0.005$.

These thresholds are informed by `gen_1` results and by the requirement that the Kalman filter converge to the same track parameters as with the RK4 extrapolator. A full integration test in Allen is beyond the scope of this study but is the logical next step if these criteria are met.

RQ2. Physics-informed training is considered beneficial if, at the same `hidden_dims` = [96, 96]:

1. the best PINN or RK_PINN configuration achieves lower mean position error than `mlp/small`, after accounting for the confounds listed in Sec. 9; or
2. the physics-informed model shows qualitatively better generalisation in the low-statistics tails of the parameter space (low p , large Δz , or extreme slopes).

8 Analysis Plan

8.1 Aggregate Comparisons

1. **Summary table:** All nine models ranked by mean position error, 95th-percentile position error, mean slope error, and best epoch. Include training wall-clock time per epoch.
2. **MLP scaling plot:** Mean and 95th-percentile position error vs. parameter count for the three MLP sizes.
3. λ_{pde} **sensitivity:** Position error vs. λ_{pde} for PINN and RK_PINN families.

8.2 Δz -Binned Analysis

To assess performance across the full propagation range, the test set will be binned in Δz :

- [25, 100] mm (VELO inter-sensor),
- [100, 500] mm (short-range),
- [500, 2000] mm (medium-range),
- [2000, 5000] mm (long-range),
- [5000, 10000] mm (full-detector).

For each bin: mean, 95th percentile, and 99th percentile of $\|\Delta \mathbf{r}\|$ and $\|\Delta \mathbf{s}\|$. This is essential to verify that the extended Δz coverage (`gen_2`) does not degrade short- Δz performance relative to `gen_1`.

8.3 Momentum-Binned Analysis

Low-momentum tracks ($p < 5 \text{ GeV}/c$) experience the strongest magnetic deflection and are most challenging for the network. The test set will be binned in p :

- [1, 5] GeV/c (strongly bent),
- [5, 20] GeV/c (moderately bent),
- [20, 200] GeV/c (weakly bent).

8.4 Diagnostic Plots

The following visualisations will be produced for each model:

1. Training and validation loss curves (total, and per-component for PINN/RK_PINN).
2. 2D heatmap of $\|\Delta \mathbf{r}\|$ in the $(\Delta z, p)$ plane.
3. Scatter plot of predicted vs. true x_f and $t_{x,f}$ for a random subsample.
4. Residual distributions Δx , Δy , Δt_x , Δt_y (histograms with Gaussian fits).
5. Jacobian Frobenius norm vs. Δz (binned).
6. RK_PINN stage weights evolution during training (initialised at $[1, 2, 2, 1]/6$; do they remain near RK4?).
7. PINN IC loss over training (should remain near zero if residual formulation works correctly).

8.5 Gen_1 vs. Gen_2 Comparison

The `mlp/tiny` configuration [64, 64] is shared between `gen_1` and `gen_2` (though trained on different data). Direct comparison of position and slope errors will quantify the impact of extending Δz coverage. The `gen_1` baseline values are:

Metric	gen_1	gen_2
$\langle \ \Delta \mathbf{r}\ \rangle$ (mm)	0.491	(this study)
$\ \Delta \mathbf{r}\ _{95\%}$ (mm)	1.452	(this study)
$\langle \ \Delta \mathbf{s}\ \rangle$	0.00491	(this study)
Best epoch	105	(this study)

9 Known Limitations and Confounds

This section catalogues the known limitations of the experimental design. These must be considered when interpreting results.

1. **Activation function mismatch.** MLPs use SiLU; PINN and RK_PINN use tanh. The tanh choice is motivated by the requirement for smooth second-order derivatives in the PDE loss (SiLU’s second derivative has a discontinuity at $x \rightarrow -\infty$ in the limit). However, SiLU generally outperforms tanh for regression tasks due to its non-saturating positive branch [7]. If a physics-informed model outperforms the MLP, the improvement cannot be unambiguously attributed to the physics loss; conversely, if the MLP outperforms, the physics loss may be beneficial but masked by the inferior activation. A controlled comparison would require running the MLP with tanh activation.
2. **Batch size mismatch.** MLPs train with batch size 4096; PINN and RK_PINN use 1024 due to the memory overhead of the physics loss computation (8 collocation points $\times$ forward pass $\times$ 4 autograd calls). At a training set of 40e6 samples, one epoch comprises ~ 9766 steps at batch size 4096 vs. ~ 39063 steps at batch size 1024. The physics-informed models therefore see $4\times$ more optimiser updates per epoch, which may independently affect convergence.
3. **Learning rate mismatch.** MLPs use $\text{lr} = 10^{-3}$; PINN/RK_PINN use 5×10^{-4} . The lower rate was chosen for training stability when balancing multiple loss terms, but it constitutes an uncontrolled variable.
4. **Parameter count mismatch (RK_PINN).** As shown in Table 1, the RK_PINN at `hidden_dims = [96, 96]` has ~ 40000 parameters (~ 156 kB), approximately $4\times$ the MLP and PINN at the same hidden dimensions. This makes direct accuracy-per-parameter comparison misleading. The RK_PINN results are primarily of scientific interest (RQ2) and cannot address RQ1 (GPU deployment) without architectural changes.
5. **PINN training cost.** The PDE loss with 8 collocation points requires ~ 8 additional forward passes and 32 autograd calls per batch. PINN epochs take ~ 50 min vs. ~ 7 min for the MLP on a V100. Per-epoch training cost differs by approximately $7\text{--}8\times$.
6. **No explicit RK4 timing baseline.** The ground truth is the RK4 output, so the “baseline error” is zero by definition. The relevant question is whether the NN error is small enough for the downstream Kalman filter, which requires an integration test in Allen that is beyond the scope of this protocol.
7. **Single polarity.** All data use MagDown ($q = -1$). MagUp data are not included. The field map symmetry $B_y(x, y, z; \text{MagUp}) = -B_y(x, y, z; \text{MagDown})$ implies that a model trained on MagDown should work for MagUp with a sign flip of q/p , but this is not tested.

8. **No material interactions.** The RK4 integrator conserves momentum ($dq/p/dz = 0$). Real tracks lose energy and scatter in detector material. The trained models learn vacuum propagation only.
9. **Jacobian metric sensitivity.** The Jacobian Frobenius norm $\|J\|_F$ showed poor discriminative power in `gen_1`, with all models yielding similar values. This may be because $\|J\|_F$ is dominated by the $\partial x_f/\partial x_0 \approx 1$ and $\partial y_f/\partial y_0 \approx 1$ diagonal elements at small Δz , while the physically interesting off-diagonal elements (e.g., $\partial x_f/\partial(q/p)$) are comparatively small.

10 Reproducibility

10.1 Random Seeds

A single global seed (42) controls all sources of randomness: `random.seed`, `numpy.random.seed`, `torch.manual_seed`, and `torch.cuda.manual_seed_all`. Additionally, `torch.backends.cudnn.deterministic` is set to `True` and `torch.backends.cudnn.benchmark` to `False`. Data generation uses per-batch seeds as specified in Eq. (10).

10.2 File Manifest

All files required to reproduce this experiment are contained within `experiments/gen_2/`:

Path	Contents
<code>configs/mlp/*.json</code>	MLP configurations (3 files)
<code>configs/pinn/*.json</code>	PINN configurations (3 files, legacy)
<code>configs/rk_pinn/*.json</code>	RK-PINN configurations (3 files, legacy)
<code>configs/v2_fixes/pinn_v2/*.json</code>	PINN_v2 configs (3 files, 2026-04-21)
<code>configs/v2_fixes/neural_rk4/*.json</code>	NeuralRK4 configs (3 files, 2026-04-21)
<code>models/train.py</code>	Training script (Fix F added 2026-04-21)
<code>models/architectures.py</code>	Model definitions incl. PINN_v2, NeuralRK4
<code>utils/rk4_propagator.py</code>	RK4 integrator
<code>utils/magnetic_field.py</code>	Unified field module
<code>data/merge_batches.py</code>	Batch merger
<code>condor/submit_datagen.sub</code>	Data generation jobs
<code>condor/run_datagen.sh</code>	Data generation worker
<code>condor/submit.sub</code>	Original training jobs (9 entries)
<code>condor/submit_v2_fixes.sub</code>	v2 fixes training jobs (6 entries)
<code>condor/run_training.sh</code>	Training worker
<code>condor/jobs.txt</code>	Original job list
<code>condor/jobs_v2_fixes.txt</code>	v2 fixes job list
<code>ISSUES_WITH_PINNS_AND_RK_PINNS.md</code>	Audit + resolution report (2026-04-21)

10.3 Software Environment

The full environment specification is frozen in:

- `environment.yml` — Conda environment with all packages pinned to exact versions,
- `requirements.txt` — pip-installable subset with version pins.

Key versions: Python 3.10, PyTorch 2.9.1+cu128, NumPy 2.2.6, MLflow 3.10.1, SciPy 1.15.3.

10.4 MLflow Tracking

All training runs are logged to the local MLflow tracking server under experiment `gen_2_track_extrapolation`. The MLflow database is stored at `experiments/gen_2/mlruns/`. Run artefacts include the best model checkpoint, training configuration, per-epoch metrics, and test-set evaluation results.

11 Architectural Fixes: PINN_v2 and NeuralRK4 (2026-04-21 Revision)

After the initial `gen_2` submission completed, a theoretical audit (companion document `experiments/gen_2/ISSU`) identified three architectural pathologies that prevented the legacy PINN and RK_PINN models (Secs. 4.2, 4.3) from converging to a useful operating point on their own terms. This section documents the fixes applied on 2026-04-21 and the two replacement architectures — `PINN_v2` and `NeuralRK4` — that now supersede the original PINN and RK_PINN in the `gen_2` submission.

11.1 Diagnosed Pathologies

1. **PINN is structurally linear in $\zeta = z/\Delta z$.** The legacy PINN envelope (Eq. 18), $\hat{\mathbf{y}}(\zeta) = \mathbf{y}_{\text{base}}(\zeta) + \zeta \boldsymbol{\delta}_{\boldsymbol{\theta}}(\mathbf{y}_0)$, contains no ζ -dependent nonlinearity because the encoder is fed only the initial state $\mathbf{y}_0$. The trajectory class is therefore affine in ζ ; since the Lorentz ODE is nonlinear in $\mathbf{y}$ and nonlinear in z through $\mathbf{B}(x, y, z)$, the PDE residual (Eq. 23) has a *structural lower bound* that the optimiser cannot reduce. Empirically, the PINN λ_{pde} response was non-monotonic.
2. **PDE residual cost was $N_c \times 4$ reverse-mode backwards per batch.** With $N_c = 8$ collocation points and four output components, the legacy implementation performed 32 `torch.autograd.grad` calls per batch. Because the input ζ is one-dimensional and the output four-dimensional, the Jacobian is 4×1 and forward-mode automatic differentiation (JVP) recovers all four components in a single sweep.
3. **RK_PINN was not a Runge–Kutta method.** The softmax-weighted combination (Eq. 28) averages *states* evaluated at $\zeta \in \{0.25, 0.5, 0.75, 1.0\}$, not *derivatives*. Classical RK4 averages derivatives of a single propagating state with fixed Butcher weights $(1, 2, 2, 1)/6$ to produce one endpoint state; a convex combination of states at different ζ has no approximation-theoretic justification. Combined with the `features.detach()` call in the physics branch, this produced slope errors 43–587 $\times$ worse than the MLP.

The proposed fixes (labels follow the audit document) are:

Label	Change	Priority
A	Replace <code>autograd.grad</code> loop with <code>torch.func.jvp</code>	P0
B	Stochastic collocation: n_c MC draws $\zeta \sim \mathcal{U}(0, 1)$ per sample	P0
C	Give PINN a ζ -dependent trunk	P0
D'	Rewrite RK-style model as neural ODE with RK4 integrator over exact Lorentz RHS	P1
E	Remove <code>features.detach()</code> in <code>RK_PINN.compute_physics_loss</code>	P0
F	Linear warm-up $\lambda_{\text{pde,ic}}(t) = \lambda^* \min(1, (t+1)/5)$	P1

11.2 PINN_v2 — Fixes A + B + C + F

The corrected envelope is

$$\hat{\mathbf{y}}_{\boldsymbol{\theta}(\zeta)} = \mathbf{y}_{\text{base}}(\zeta) + \zeta \cdot \text{Head}\left(\text{Enc}\left([\tilde{\mathbf{y}}_0; \zeta]\right)\right), \quad (34)$$

where $\mathbf{y}_{\text{base}}$ is still the straight-line baseline (Eq. 19) and $[\tilde{\mathbf{y}}_0; \zeta] \in \mathbb{R}^6$ replaces the $\mathbb{R}^5$ input of the legacy encoder (Eq. 20). Because the ζ -prefactor still multiplies the entire correction, $\hat{\mathbf{y}}_{\boldsymbol{\theta}(0) \equiv \mathbf{y}_0}$ exactly and the IC loss (Eq. 22) remains identically zero by construction. The trajectory is now a smooth nonlinear function of ζ , and the structural floor on $\mathcal{L}_{\text{pde}}$ is eliminated.

The PDE residual (Eq. 23) is evaluated with a single forward-mode JVP per Monte-Carlo collocation draw:

$$(\hat{\mathbf{y}}(\zeta_i), \frac{\partial \hat{\mathbf{y}}}{\partial \zeta} \Big|_{\zeta_i}) = \text{torch.func.jvp}(\text{forward_at_z}, (\zeta_i,), (1,)), \quad \zeta_i \sim \mathcal{U}(0, 1), \quad (35)$$

with n_c independent draws per training sample (default $n_c = 2$). This replaces $N_c \times 4$ reverse-mode backwards with a single JVP sweep per draw; CPU microbenchmarks show a $\sim 2\times$ per-iteration speed-up even before combining with the reduced n_c , and a $\sim 30\times$ reduction in PDE-term cost overall. The data and IC losses retain the non-dimensionalisation of Eqs. (17) and (22); the PDE residual uses the same $\sigma_k/\Delta z$ scaling as Eq. (24). A linear warm-up (Fix F),

$$\lambda_{\text{pde}}(t) = \lambda^* \cdot \min(1, (t+1)/T_{\text{warm}}), \quad T_{\text{warm}} = 5 \text{ epochs}, \quad (36)$$

prevents the early-epoch data/physics conflict that caused the legacy PINN to early-stop at epoch 10–16.

Parameter count. PINN_v2 with `hidden_dims = [256, 256]` has 68612 parameters, dominated by the wider trunk required to reach the PDE-residual floor of the fully nonlinear model class.

11.3 NeuralRK4 — Fix D'

NeuralRK4 replaces the RK_PINN state-mixture with a genuine fourth-order Runge–Kutta integrator of the Lorentz ODE, perturbed by a small learned correction to the right-hand side:

$$\mathbf{f}_{\boldsymbol{\theta}(\mathbf{y}, z; q/p)} = \mathbf{F}(\mathbf{y}, z; q/p) + e^\alpha (\boldsymbol{\sigma}_y / \Delta z) \odot \mathbf{g}_\phi(\mathbf{y}, z; q/p), \quad (37)$$

where $\mathbf{F}$ is the exact Lorentz RHS (Eqs. 11–14), $\mathbf{g}_\phi$ is a small MLP, and α is a single learnable log-scale initialised at $\log(10^{-3})$ so that training begins at “pure RK4 plus an infinitesimal drift.” The $\boldsymbol{\sigma}_y / \Delta z$ factor brings the learned correction into the physical scale of the RHS. The state update is

$$\mathbf{y}_{i+1} = \mathbf{y}_i + \frac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4), \quad \mathbf{k}_m = \mathbf{f}_{\boldsymbol{\theta}(\mathbf{y}_i^{(m)}, z_i^{(m)}; q/p)}, \quad h = \Delta z / N_{\text{RK}}, \quad (38)$$

iterated $N_{\text{RK}} \in \{1, 2\}$ times with the standard RK4 nodes $\mathbf{y}_i^{(m)}$. Butcher weights are *fixed* (not learned) and multiply *derivatives*, restoring the approximation-theoretic meaning the original RK_PINN lacked (Sec. 11.1 (3)).

Because the physics is baked into the integrator, no auxiliary PDE or IC loss is required: the training loss is the same component-weighted endpoint MSE as the MLP (Eq. 17), $\lambda_{\text{pde}} = \lambda_{\text{ic}} = 0$.

Parameter count. NeuralRK4 with `hidden_dims = [64, 64]` has 4869 trainable parameters (matched to `mlp/tiny`); with `hidden_dims = [128, 128]` it has 17925. The N_{RK} choice does not change the parameter count, only the integration granularity.

11.4 Measured Outcome on train_50M_dz25.npz

All v2 runs share the training data, split (80/10/10), seed (42), batch size (1024 for PINN_v2 and `neural_rk4/small_2step`; 2048 otherwise), base learning rate (5×10^{-4}), and cosine annealing schedule of Sec. 6. The MLP baselines were trained on the same dataset under the schedule of Table 2. Table 3 summarises the test-set metrics; two v2 runs are still in progress at the time of this edit and are marked with the latest validation-log value.

Table 3: Final test metrics for the `gen_2 v2` submission (cluster 4262459). MLP baselines from cluster 4262446 on the same `train_50M_dz25.npz` dataset included for comparison. All quantities computed on the 5M-sample test split. Starred entries are the latest value parsed from the training log for the two runs still in progress.

Family	Config	Hidden	Params	Pos [mm]	Slope [rad]	best ep / total	Wall [h]
MLP	tiny	[64, 64]	4868	0.3868	8.5×10^{-5}	62 / 64	3.69
MLP	small	[128, 128]	18692	0.5092	1.21×10^{-4}	16 / 46	2.57
MLP	medium	[256, 256, 128]	102788	0.3046	9.5×10^{-5}	20 / 40	1.63
MLP	large	[512, 512, 256]	401412	0.3798	7.6×10^{-5}	14 / 44	1.81
MLP	wide	[512, 512, 256, 128]	434180	0.3139	7.9×10^{-5}	24 / 42	1.87
PINN_v2	$\lambda = 0.1$	[256, 256]	68612	0.2352	6.8×10^{-5}	4 / 41	7.84
PINN_v2	$\lambda = 1.0$	[256, 256]	68612	0.3218	8.6×10^{-5}	1 / 41	7.87
PINN_v2	$\lambda = 10.0$	[256, 256]	68612	$\sim 0.40^*$	$1.5 \times 10^{-4*}$	—	—
NeuralRK4	tiny, 1 step	[64, 64]	4869	0.1339	5.0×10^{-5}	40 / 42	3.62
NeuralRK4	small, 1 step	[128, 128]	17925	0.1251	5.0×10^{-5}	40 / 42	3.64
NeuralRK4	small, 2 steps	[128, 128]	17925	$\sim 0.128^*$	$5.0 \times 10^{-5*}$	—	—

Before/after contrast. The pre-fix results on an earlier cluster (4217847, variable- Δz dataset) gave RK_PINN $\times 10^{-2}$, 1.7×10^{-1}] rad and a non-monotone PINN λ_{pde} response (see ISSUES_WITH_PINNS_AND_RK_PINNS.md §2). On the v2 submission:

1. *Slope catastrophe eliminated.* NeuralRK4 achieves 5.0×10^{-5} rad — a three-orders-of-magnitude improvement over the legacy RK_PINN — because Butcher weights multiply the correct quantity (derivatives) and the backbone is no longer detached from the physics gradient.
2. *Monotone λ_{pde} response recovered.* PINN_v2 position degrades monotonically as λ_{pde} increases ($0.235 \rightarrow 0.322 \rightarrow \sim 0.40$ mm), consistent with a genuine data/physics trade-off rather than the structural-floor pathology of the legacy PINN.
3. *MLPs beaten on accuracy per parameter.* At 4869 parameters (matched to `mlp/tiny`), NeuralRK4 reaches 0.134 mm, $2.3\times$ better than the best MLP (`mlp/medium`, 102788 parameters, 0.305 mm) with $\sim 21\times$ fewer parameters. The gains at 17925 parameters are similar: 0.125 mm vs. the best MLP’s 0.305 mm.

Known limitations carried over. Neither PINN_v2 nor NeuralRK4 uses the per-sample initial z_0 ; both assume $z_0 = 0$. The training data has $z_0 \in [0, 14000]$ mm and the magnetic field is strongly z -dependent, so this is an explicit modelling approximation, flagged in the source with `FIX NOTE` comments and documented in ISSUES §7 and §9.4 for resolution in `gen_3`.

11.5 Provenance and Reproducibility

The fixes are traceable end-to-end:

- **Code.** `experiments/gen_2/models/architectures.py` — classes `PINN_v2` (added 2026-04-21, fixes A+B+C) and `NeuralRK4` (added 2026-04-21, fix D’) with the original PINN and RK_PINN kept for archival comparison. `MODEL_REGISTRY` now exposes `pinn_v2` and `neural_rk4`.
- **Training script.** `experiments/gen_2/models/train.py` — added `physics_warmup_epochs` (default 5) and a per-epoch ramp (Fix F).
- **Configs.** `experiments/gen_2/configs/v2_fixes/pinn_v2/*.json` (3 files, $\lambda_{pde} \in \{0.1, 1.0, 10.0\}$) and `configs/v2_fixes/neural_rk4/*.json` (3 files, `tiny_1step`, `small_1step`, `small_2step`). Each config records the list `arch_fixes` of applied fixes and the `supersedes.experiment` identifier of the legacy run it replaces.

- **Cluster.** `condor/submit_v2_fixes.sub` with `condor/jobs_v2_fixes.txt` dispatches the six v2 jobs. Submitted as cluster 4262459 on 2026-04-21; legacy PINN/RK_PINN jobs (cluster 4262446, ProcIds 5–10) were cancelled at the same time.
- **MLflow.** Experiment name `gen_2_track_extrapolation_v2_fixes`, stored in `experiments/gen_2/mlruns` disjoint from the `gen_2_track_extrapolation_true_gen2_data` experiment that holds the MLP baselines.

12 Companion Reports

This protocol is complemented by three standalone results reports stored alongside this document in `docs/reports/`. They are written to be read sequentially; each references but does not duplicate the preceding material.

- `gen1_results.tex` — Initial MLP-only feasibility study on the V1 dataset (`experiments/gen_1/V1/data.g`). Motivates the whole project (64 kB CUDA constant-memory budget, Allen HLT1 integration), characterises the `twodip.rtf` magnetic field, presents the 17-MLP architecture sweep, and establishes the Pareto frontier. Labels prefixed `gen1:`.
- `gen2_old_data_results.tex` — First PINN and RK_PINN study, run with the `gen_2` codebase on the V1 dataset (HTCondor cluster 4217847). Diagnoses the four pathologies (P1 affine-in- ζ trunk, P2 reverse-mode AD cost, P3 state-averaging RK_PINN, P4 softmax-drift of Butcher weights) that motivate the v2 fixes described in §11 of this protocol. Labels prefixed `gen2old:`.
- `gen2_new_data_results.tex` — v2 fixes (PINN_v2, NeuralRK4) on the new variable- Δz dataset (`train_50M_dz25.npz`), HTCondor clusters 4262446 (MLP baselines) and 4262459 (v2 fixes). Presents the final results table: NeuralRK4 at 17.9k parameters reaches 0.124 mm position error and 5.0×10^{-5} rad slope error, beating the best MLP (100.9k parameters, 0.305 mm) by $2.5\times$ position and $2\times$ slope. Labels prefixed `gen2new:`.

13 Summary

This protocol defines a nine-configuration experiment comparing three neural network architectures (MLP, PINN, RK_PINN) for replacing the RK4 track extrapolator in LHCb’s Allen HLT1 trigger. The `gen_2` dataset extends the propagation range to $\Delta z \geq 25$ mm to cover VELO sensor spacing, and all MLP architectures are constrained to ≤ 64 kB for GPU deployment. Six confounding variables are identified (activation, batch size, learning rate, parameter count, training cost, and Jacobian metric sensitivity). The analysis plan specifies aggregate, Δz -binned, and momentum-binned evaluation to address both the engineering (RQ1) and scientific (RQ2) research questions.

References

- [1] R. Aaij *et al.* (LHCb Collaboration), “The LHCb detector at the LHC,” JINST **3**, S08005 (2008). doi:10.1088/1748-0221/3/08/S08005.
- [2] E. Bos and E. ”; see also M. Needham, “The LHCb track extrapolation,” LHCb-2007-140, CERN-LHCb-2007-140 (2007).
- [3] LHCb magnetic field map, internal documentation.

- [4] R. Aaij *et al.* (LHCb Collaboration), “Allen: A high-level trigger on GPUs for LHCb,” *Comput. Softw. Big Sci.* **4**, 7 (2020). doi:10.1007/s41781-020-00039-7.
- [5] R. Frühwirth, “Application of Kalman filtering to track and vertex fitting,” *Nucl. Instrum. Meth. A* **262**, 444 (1987). doi:10.1016/0168-9002(87)90887-4.
- [6] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proc. AISTATS* (2010), pp. 249–256.
- [7] S. Elfving, E. Uchibe, and K. Doya, “Sigmoid-weighted linear units for neural network function approximation in reinforcement learning,” *Neural Networks* **107**, 3 (2018). doi:10.1016/j.neunet.2017.12.012.
- [8] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. ICLR* (2019). arXiv:1711.05101.
- [9] J. Stiasny, G. S. Misyris, and S. Chatzivasileiadis, “Physics-informed neural networks for non-linear system identification applied to power system dynamics,” in *Proc. IEEE PowerTech* (2021). doi:10.1109/PowerTech46648.2021.9494769.
- [10] J. Zhai, T. Dobrowolski, and E. Bhatt, “Solving differential equations with physics-informed neural networks incorporating Runge-Kutta methods,”
- [11] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *J. Comput. Phys.* **378**, 686 (2019). doi:10.1016/j.jcp.2018.10.045.
- [12] S. Wang, Y. Teng, and P. Perdikaris, “Understanding and mitigating gradient flow pathologies in physics-informed neural networks,” *SIAM J. Sci. Comput.* **43**(5), A3055 (2021). doi:10.1137/20M1318043.
- [13] A. S. Krishnapriyan, A. Gholami, S. Zhe, R. M. Kirby, and M. W. Mahoney, “Characterizing possible failure modes in physics-informed neural networks,” in *Adv. Neural Inf. Process. Syst.* (NeurIPS), **34**, 26548 (2021).